

MixTriad: A Python toolkit for triangulating regression, gradient-boosting, and fuzzy-set QCA evidence in observational social-media research

First A. Author^a, Second B. Author^b · ^aAffiliation One · ^bAffiliation Two · corresponding: author@institution.edu

Abstract. Empirical studies of online virality, technology adoption, and comparable observational phenomena increasingly triangulate three kinds of evidence—parametric regression, machine-learning feature importance, and configurational set-theoretic analysis—yet no open tool chains these stages together, and fuzzy-set Qualitative Comparative Analysis (fsQCA) in particular lacks a maintained, importable Python implementation. MixTriad packages the full three-stage workflow into one reproducible pipeline: (i) OLS on the log outcome with a negative-binomial robustness model, (ii) Bayesian-optimized gradient boosting under a leakage-safe repeated hold-out protocol with permutation-importance screening, and (iii) a complete fsQCA engine covering direct calibration, necessity analysis, truth-table construction, and Quine–McCluskey minimization with conservative, parsimonious, and intermediate solutions. A single YAML file or a five-line script reproduces every table and figure of a virality study of AI-generated misinformation short videos, and the same commands transfer unchanged to any tabular dataset with a continuous or count outcome, making the toolkit reusable across communication research, information systems, management, and public-health studies.

Keywords: fsQCA; mixed methods; gradient boosting; permutation importance; misinformation; Python

Table 1. Code metadata

Nr	Code metadata description	Value
C1	Current code version	v1.0.0
C2	Permanent link to code/repository used for this code version	https://github.com/ElsevierSoftwareX/SOFTX-D-26-XXXXX
C3	Permanent link to Reproducible Capsule	—
C4	Legal Code License	MIT
C5	Code versioning system used	git
C6	Software code languages, tools and services used	Python 3.9+; NumPy, pandas, scikit-learn, statsmodels, Matplotlib, PyYAML; optional: XGBoost, Optuna, SHAP
C7	Compilation requirements, operating environments and dependencies	Pure Python (no compilation); Linux / macOS / Windows; pip install -e ".[all]"
C8	If available, link to developer documentation/manual	README.md and docstrings in the repository
C9	Support email for questions	author@institution.edu

1. Motivation and significance

Researchers who study why some online content spreads and other content does not routinely need three complementary kinds of evidence. Regression establishes which antecedents have statistically significant net effects; tuned machine-learning models establish which antecedents matter most predictively once non-linearities and interactions are allowed; and Qualitative Comparative Analysis establishes which combinations of conditions are jointly sufficient for the outcome, acknowledging that different causal recipes can produce the same result (equifinality). The same triangulation logic appears far beyond virality research, in technology-adoption, entrepreneurship, policy-diffusion, and health-behaviour studies.

Executing this workflow today is disproportionately costly. The regression stage is typically run in R or Stata, the boosting stage in Python, and the configurational stage in the standalone fsQCA desktop program or the R QCA package; results are moved between environments by hand, calibration anchors and tuning decisions are rarely archived, and the machine-learning stage is frequently reported without a leakage-safe validation protocol. To the best of our knowledge, no openly available software provides fsQCA as an importable, unit-tested Python library, and none chains the three stages so that the conditions entering the configurational analysis are selected transparently from the importance ranking of the predictive stage.

MixTriad addresses this gap. It implements the complete fsQCA workflow in pure Python—direct calibration with the log-odds transformation, necessity analysis, truth-table construction with consistency, PRI, and frequency cut-offs, and Boolean minimization by the Quine–McCluskey algorithm yielding conservative, parsimonious, and intermediate solutions with core/peripheral condition classification—and couples it to a parametric stage (OLS with heteroskedasticity-robust errors plus a negative-binomial robustness model) and a predictive stage (Bayesian-optimized gradient boosting evaluated by five-times-repeated fixed-seed hold-outs, with per-repetition hyperparameter search confined to the training fold). The contribution is non-trivial in two respects: the fsQCA engine fills a long-standing hole in the Python scientific ecosystem, and the pipeline design turns an ad-hoc, three-environment practice into a single declarative configuration whose every intermediate artefact is written to disk for audit.

The software was developed alongside an empirical study of the virality of AI-generated misinformation short videos on TikTok and YouTube Shorts, in which eleven antecedents organized by information-ecology theory were analysed with exactly this three-stage design. That study is the illustrative example of Section 3; the toolkit itself is domain-agnostic.

2. Software description

2.1 Software architecture

MixTriad is implemented in Python (≥ 3.9) and organized into six modules (Fig. 1). The data module declares variable roles through a Schema object (outcome, antecedents, controls, log-transformed and categorical variables, optional theory-dimension labels) and provides a Krippendorff's-alpha routine for hand-coded ordinal antecedents. The regression, boosting, and fsqca modules implement the three analytical stages and are independently importable. The pipeline module orchestrates the stages, writes every table as tidy CSV, renders publication-ready figures, and consolidates a Markdown report; the cli module exposes the whole workflow as a single command, `mixtriad run config.yaml`. Core dependencies are limited to the standard scientific stack (NumPy, pandas, scikit-learn, statsmodels, Matplotlib); XGBoost, Optuna, and SHAP are optional extras, and in their absence the boosting stage degrades gracefully to scikit-learn's histogram gradient boosting with random search, so the pipeline never hard-fails on a missing accelerator.

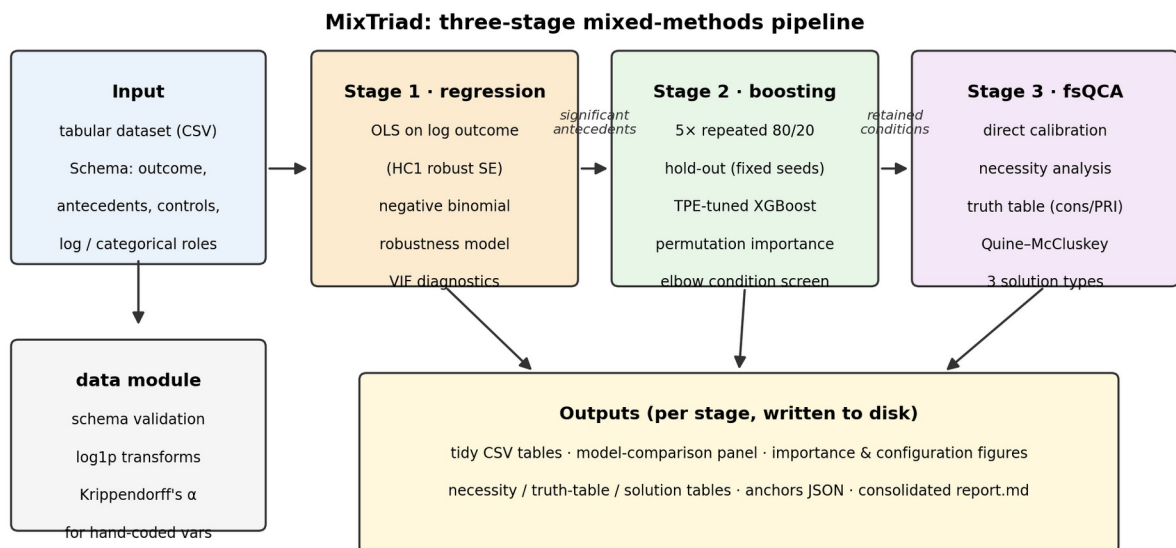


Fig. 1. Architecture and data flow of MixTriad. Each stage hands a documented artefact to the next: significant antecedents (Stage 1), the importance-screened condition set (Stage 2), and calibrated set memberships (Stage 3).

2.2 Software functionalities

Stage 1 fits OLS on the log-transformed outcome with HC1 robust standard errors, an NB2 negative-binomial model on the raw count as a robustness check, and reports variance-inflation factors; all results are returned as tidy data frames with significance stars.

Stage 2 enforces a leakage-safe protocol adapted from the accompanying study: five repetitions of an 80/20 hold-out with fixed seeds (42, 123, 456, 789, 1024); within each training fold, tree-structured Parzen-estimator search (via Optuna) minimizes five-fold cross-validated RMSE over the number of boosting rounds (100–1000), depth (3–10), learning rate (0.01–0.30), and row/column subsampling (0.5–1.0); test folds are never touched during tuning. The stage outputs a model-comparison panel against OLS, default random-forest, and default gradient-boosting baselines, plus permutation importance averaged over held-out folds and an automatic elbow cut-off that screens the antecedents forwarded to Stage 3 (a user-supplied cut-off overrides the elbow).

Stage 3 provides the full fsQCA vocabulary as plain functions: `direct_calibrate()` maps raw variables to fuzzy memberships from three anchors (10th/50th/90th percentiles are offered as defaults); `necessity_table()` reports necessity consistency and coverage for every condition and its negation; `build_truth_table()` assigns cases to property-space corners and applies consistency, PRI, and frequency cut-offs; `minimize()` returns conservative, parsimonious, and intermediate solutions—the last using directional expectations to admit easy counterfactuals only—each scored with recipe-level consistency, raw and unique coverage, and solution-level statistics; `core_periphery()` classifies conditions for standard configuration charts, which the pipeline renders automatically for both the outcome and its negation. Calibration anchors are archived to JSON so that every set-membership decision is reproducible.

All functionality is accessible at three levels—individual functions, per-stage runners, and the one-call `Pipeline.run_all()`—so the toolkit serves both interactive exploration and batch replication.

3. Illustrative examples

The repository ships a 388-row synthetic corpus whose schema mirrors the misinformation-video study: eleven antecedents across the four information-ecology dimensions (actor: creator reach, creator activity, audience youthfulness; content: topic category, duration, headline length, hashtag count, emotional intensity; technology: visual realism, audio-visual coherence; environment: account topic focus), a publication-age control, and a heavy-tailed engagement outcome. No platform data are redistributed; the generator encodes the qualitative structure reported in the study, including a planted reach $\times$ topic-focus $\times$ emotion configuration. The complete analysis is one command:

```
python examples/make_dataset.py
mixtriad run examples/misinfo_config.yaml
```

or, equivalently, five lines of Python:

```
from mixtriad import Pipeline, FsqaSpec, Schema
df = pd.read_csv("examples/misinfo_videos_synthetic.csv")
schema = Schema(outcome="engagement", antecedents=[...11 vars...],
                controls=["publication_age"], log_transform=["creator_reach", ...])
Pipeline(df, schema, "out").run_all(FsqaSpec(directional={"creator_reach": 1, ...}))
```

Stage 1 recovers the expected net effects (OLS $R^2 = 0.757$ on the log outcome), with creator reach, account topic focus, and emotional intensity strongly significant. Stage 2's honest out-of-sample panel is reproduced in Table 2: relative to OLS, the tuned model lowers test RMSE by 18% and raises R^2 from 0.70 to 0.80, confirming material non-linear and interaction structure. The permutation-importance figure written by the pipeline (Fig. 2) shows the automatic elbow screen retaining 4 antecedents (creator_reach, account_topic_focus, emotional_intensity, video_duration) for the configurational stage.

Table 2. Out-of-sample comparison written by Stage 2 (means over five fixed-seed 80/20 hold-outs; log outcome)

Model	RMSE	MAE	R^2
OLS regression	1.13	0.90	0.70
Random forest (default)	1.06	0.83	0.74

Model	RMSE	MAE	R ²
Gradient boosting (default)	1.01	0.78	0.76
XGBoost (Bayesian-tuned)	0.93	0.74	0.80

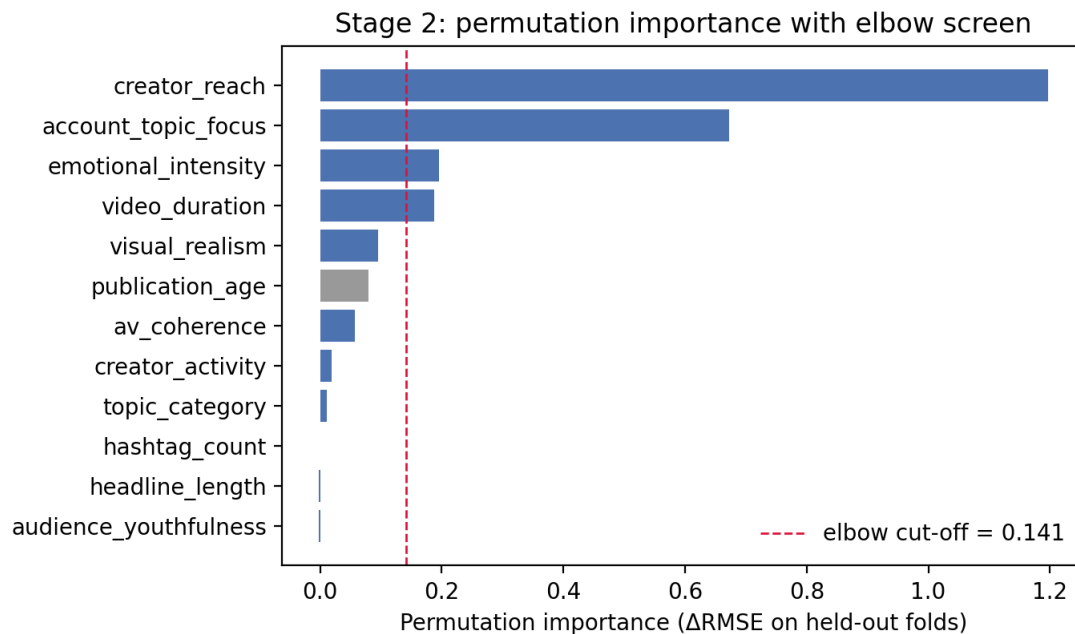


Fig. 2. Stage-2 permutation importance on held-out folds with the automatic elbow cut-off (grey bar: control variable).

Stage 3 calibrates the retained conditions from percentile anchors (archived to JSON), reports necessity, and minimizes the truth table for both high and low virality. Table 3 lists the intermediate-solution recipes for high virality (solution consistency 0.908, coverage 0.530); the pipeline's configuration chart (Fig. 3) displays the same recipes in standard core/peripheral notation. The planted combination of broad creator reach, high account topic focus, and strong emotional intensity is recovered as the core sufficient path, and the negated-outcome analysis returns the mirror-image recipe for low virality—reproducing, on synthetic data, the equifinal structure reported in the empirical study.

Table 3. Intermediate solution for high virality (Stage-3 output)

Recipe	Consistency	Raw coverage	Unique coverage
creator_reach × account_topic_focus × ~video_duration	0.951	0.388	0.1
creator_reach × account_topic_focus × emotional_intensity	0.945	0.363	0.072
creator_reach × emotional_intensity × ~video_duration	0.918	0.365	0.082

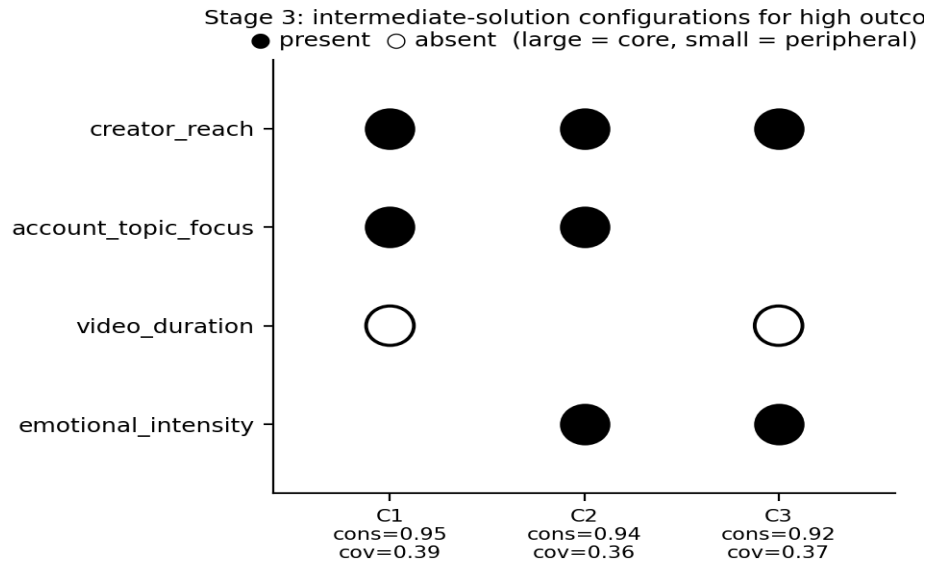


Fig. 3. Stage-3 configuration chart for high virality (● present, ○ absent; large markers = core conditions shared with the parsimonious solution).

4. Impact

MixTriad changes the day-to-day practice of mixed-methods observational research in three ways. First, it removes the three-environment hand-off: an analysis that previously required R or Stata for regression, a separate Python session for boosting, and the standalone fsQCA program for the configurational stage now runs from one configuration file, cutting the manual effort from days of tool-switching and re-entry to minutes while eliminating transcription errors between stages. Second, it hardens two chronically weak links: the boosting stage's fixed-seed, tuning-inside-the-fold protocol prevents the optimistic bias that arises when hyperparameters see the test data, and the fsQCA stage archives calibration anchors, cut-offs, and every truth-table row, making set-theoretic results auditable and exactly replicable—properties the desktop GUI workflow cannot offer.

Third, and most broadly, the toolkit lowers the entry barrier to configurational thinking for the Python-native research community. Because the fsQCA engine consists of ordinary importable functions with a pytest suite (including textbook Quine–McCluskey cases and a planted-configuration recovery test), it can be embedded in notebooks, teaching materials, and larger platform-governance or computational-social-science pipelines. The input is any tidy table with declared variable roles, so the same commands transfer to adoption studies in information systems, entrepreneurship and management research, policy-diffusion analysis, and epidemiological risk-configuration work; the misinformation-virality corpus is simply the shipped demonstration. Within our own programme, the toolkit generated every quantitative table and figure of the accompanying virality study and is being reused for a follow-up cross-platform comparison; the synthetic-data generator additionally allows methods courses to teach the full pipeline without redistributing platform data.

5. Conclusions

MixTriad packages a widely needed but previously fragmented evidence chain—net effects, honest predictive importance, and equifinal configurations—into one tested, scriptable Python pipeline, and contributes the first importable open-source fsQCA implementation for the Python ecosystem. The software is released under the MIT licence with a runnable example, a unit-test suite, and archived intermediate artefacts for full reproducibility. Future releases will add clustered and panel extensions to the regression stage, SHAP-based interaction reports, temporal QCA variants, and an importer for existing fsQCA .csv calibration files.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] B. Nardi, V. O'Day, *Information Ecologies: Using Technology with Heart*, MIT Press, Cambridge, MA, 1999.
- [2] C.C. Ragin, *Redesigning Social Inquiry: Fuzzy Sets and Beyond*, University of Chicago Press, Chicago, 2008.
- [3] S. Vosoughi, D. Roy, S. Aral, The spread of true and false news online, *Science* 359 (2018) 1146–1151.
- [4] T. Chen, C. Guestrin, XGBoost: a scalable tree boosting system, in: *Proc. 22nd ACM SIGKDD*, 2016, pp. 785–794.
- [5] T. Akiba, S. Sano, T. Yanase, T. Ohta, M. Koyama, Optuna: a next-generation hyperparameter optimization framework, in: *Proc. 25th ACM SIGKDD*, 2019, pp. 2623–2631.
- [6] F. Pedregosa, et al., Scikit-learn: machine learning in Python, *J. Mach. Learn. Res.* 12 (2011) 2825–2830.
- [7] S. Seabold, J. Perktold, Statsmodels: econometric and statistical modeling with Python, in: *Proc. 9th Python in Science Conf.*, 2010, pp. 92–96.
- [8] A. Duşa, *QCA with R: A Comprehensive Resource*, Springer, Cham, 2019.
- [9] P.C. Fiss, Building better causal theories: a fuzzy set approach to typologies in organization research, *Acad. Manage. J.* 54 (2011) 393–420.
- [10] K. Krippendorff, Computing Krippendorff's alpha-reliability, *Univ. of Pennsylvania Working Paper*, 2011.
- [11] W.J. Brady, J.A. Wills, J.T. Jost, J.A. Tucker, J.J. Van Bavel, Emotion shapes the diffusion of moralized content in social networks, *Proc. Natl. Acad. Sci.* 114 (2017) 7313–7318.
- [12] G. Pennycook, D.G. Rand, The psychology of fake news, *Trends Cogn. Sci.* 25 (2021) 388–402.
- [13] E.W. Quine, The problem of simplifying truth functions, *Am. Math. Mon.* 59 (1952) 521–531.
- [14] E.J. McCluskey, Minimization of Boolean functions, *Bell Syst. Tech. J.* 35 (1956) 1417–1444.